

Setor de Educação Profissional e Tecnológica

Tecnologia em Análise e Desenvolvimento de Sistemas

DS141 - Estruturas de Dados II

11 - Grafos não Direcionados

Prof. Alex Kutzke

Julho de 2021

Tradução adaptada dos slides presentes em [1]. Imagens e exemplos também retirados do mesmo material.

[1] <<https://algs4.cs.princeton.edu/lectures/keynote/34HashTables.pdf>>

Roteiro

Introdução

Representação

Implementação

Busca em Profundidade

Busca em Largura

Componentes Conexas

Conclusão

Introdução
●○○○○○○○

Representação
○○○○○○○○○

Implementação
○○○○

Busca em Profundidade
○○○○○○○○○

Busca em Largura
○○○○○○

Componentes Conexas
○○○○

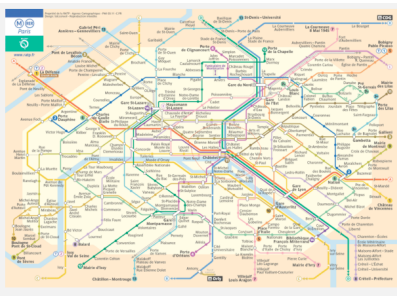
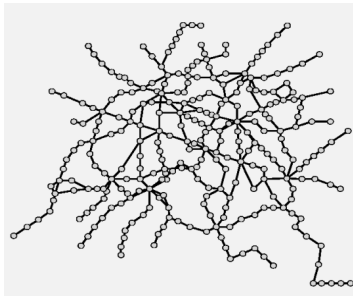
Conclusão
○○○

Introdução

Grafos

Grafos: Conjunto de **vértices** (V) conectados em pares por **arestas** (E).

$$G = (V, E)$$



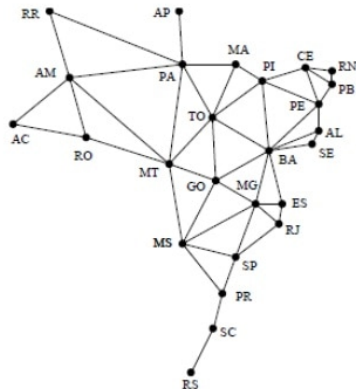
Fonte: (SEdgeWICK; WAYNE, 2014)

Grafos

Por que estudar grafos e seus algoritmos?

- Milhares de aplicações práticas;
- Vários algoritmos conhecidos;
- Abstrações interessantes e úteis;
- Campo desafiador da Ciência da Computação e da Matemática Discreta.

Fronteiras entre os estados Brasileiros



10 milhões de amizades no Facebook



Fonte: (SEdgeWICK; WAYNE, 2014)

Exemplos

Grafo	Vértice	Aresta
Comunicação	telefone, computador	cabo de fibra óptica, cabo de rede
Circuito	porta, registro, processador	fio, filamento
Financeiro	moeda, ações	transação
Transporte	interseção	rua
Jogos	posição do tabuleiro	movimento válido
Relacionamento social	pessoa	amizade
Rede neural	neurônio	sinapse
Rede proteica	proteína	interação proteína-proteína
Molécula	átomo	ligação

Alguns problemas com grafos

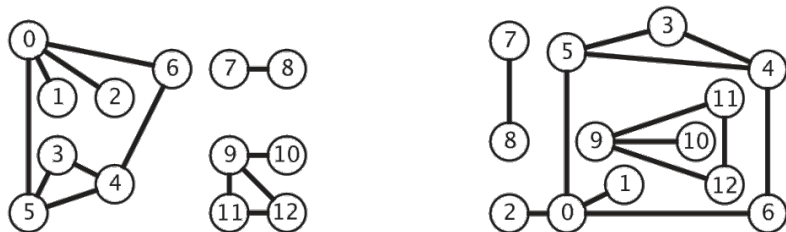
Problema	Descrição
Caminho s-t	<i>Existe um caminho entre s e t?</i>
Menor caminho s-t	<i>Qual o menor caminho entre s e t?</i>
Ciclo	<i>Existe um ciclo no grafo?</i>
Ciclo Euleriano	<i>Existe um ciclo que usa cada aresta exatamente uma vez?</i>
Ciclo Hamiltoniano	<i>Existe um ciclo que usa cada vértice exatamente uma vez?</i>
Conectividade	<i>Existe um caminho para conectar todos os vértices?</i>
Biconectividade	<i>Existe um vértice que, se removido, desconecta o grafo?</i>
Planaridade	<i>É possível desenhar o grafo em um plano sem sobrepor arestas?</i>
Isomorfismo	<i>Duas listas de adjacências representam o mesmo grafo?</i>

Pergunta Quais desses problemas são fáceis? Difíceis? Intratáveis?

Representação

Representação gráfica

Grafos podem ser representados graficamente e gerar intuições sobre a sua estrutura.



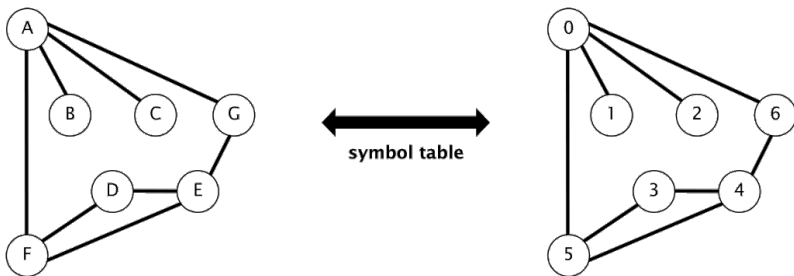
two drawings of the same graph

Fonte: (SEdgeWICK; WAYNE, 2014)

Importante Intuições podem enganar.

Vértices

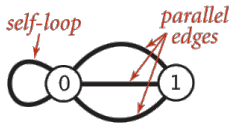
- Utilizaremos valores inteiros para nomear os vértices;
- Aplicações reais podem converter nomes e números com tabelas de símbolos.



Fonte: (SEdgeWICK; WAYNE, 2014)

Anomalias

Consideraremos os itens na figura abaixo como anomalias. Entretanto, em algumas aplicações, eles podem ser considerados importantes.



Fonte: (SEDGEWICK; WAYNE, 2014)

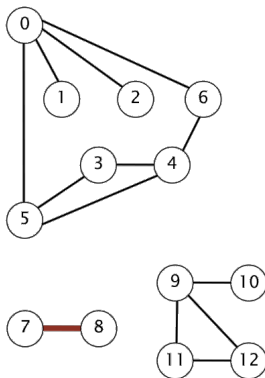
Representação Computacional

Para representar a estrutura de dados **Grafos** podemos considerar 3 opções:

1. Conjunto de arestas;
2. Matriz de adjacências;
3. Lista de adjacências;

Conjunto de arestas

Mantém um conjunto de arestas (lista ligada ou array).



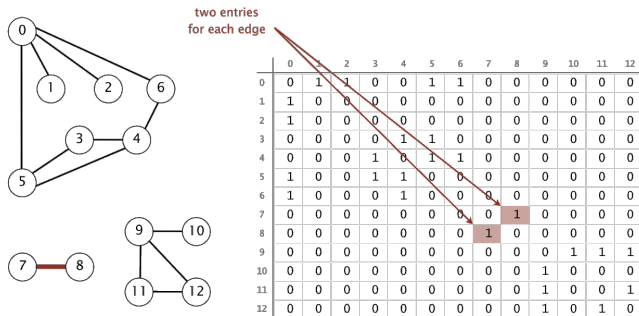
0	1
0	2
0	5
0	6
3	4
3	5
4	5
4	6
7	8
9	10
9	11
9	12
11	12

Fonte: (SEdgeWICK; WAYNE, 2014)

Q. Quanto custa para iterar sobre todos os vértices adjacentes a v ?

Matriz de Adjacências

Mantém uma matriz bidimensional de V por V booleanos;
Para cada aresta $v - w$ no grafo: $\text{adj}[v][w] = \text{adj}[w][v] = \text{true}$.

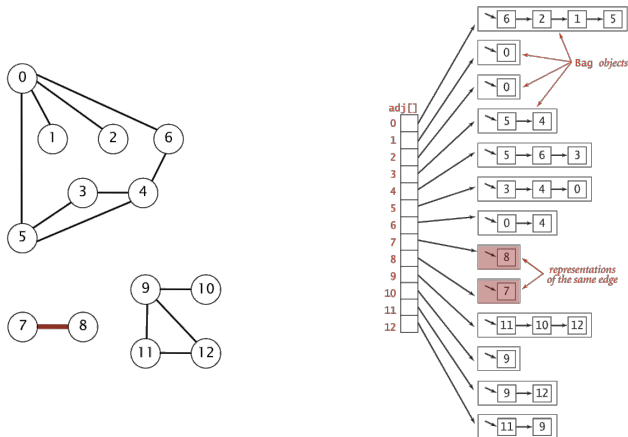


Fonte: (SEGEWICK; WAYNE, 2014)

Q. Quanto custa para iterar sobre todos os vértices adjacentes a v ?

Lista de Adjacências

Mantém um array de listas com as arestas de cada vértice.



Fonte: (SEdgeWICK; WAYNE, 2014)

Q. Quanto custa para iterar sobre todos os vértices adjacentes a v ?

Introdução
○○○○○○○○

Representação
○○○○○○○○●○

Implementação
○○○○

Busca em Profundidade
○○○○○○○○○

Busca em Largura
○○○○○○

Componentes Conexas
○○○○

Conclusão
○○○

Qual a melhor representação?

Qual a melhor representação?

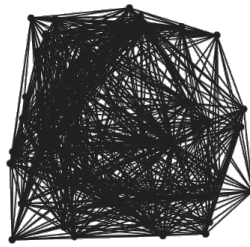
Na prática: Utilizar **lista de adjacências**:

- Algoritmos são geralmente baseados na iteração sobre os vértices adjacentes a v ;
- Grafos de problemas reais tendem a ser **esparços**:
 - Número alto de vértices mas com **grau médio** pequeno.

sparse ($E = 200$)



dense ($E = 1000$)



Two graphs ($V = 50$)

Fonte: (SEdgeWICK; WAYNE, 2014)

Comparação entre Representações

Representação	Espaço	Adicionar aresta	Aresta entre v e w ?	Iterar sobre os vizinhos de v ?
Conjunto de arestas	E	1	E	E
Matriz de Adjacências	V^2	1^*	1	V
Lista de Adjacências	$E + V$	1	$\text{grau}(v)$	$\text{grau}(v)$

* Não permite arestas paralelas

Implementação

Estrutura para Listas de Adjacências

```
1  typedef struct node *link;
2  struct node {
3      int w;                // destination node of edge
4      link next;           // next element in adjacency list
5  };
6
7  struct graph {
8      int V;                // number of vertices
9      int E;                // number of edges
10     link *adj;            // adjacency list
11 };
12 typedef struct graph *Graph;
13 typedef struct { int v; int w; } Edge;
```

Criação de um Grafo

```
1 Graph GRAPHinit(int V) {
2     int v;
3     Graph G = malloc(sizeof *G);
4     G->V = V; G->E = 0;
5     G->adj = malloc(V * sizeof(*G->adj));
6     if (G->adj == NULL){
7         fprintf(stderr, "Out of memory in GRAPHinit()\n");
8         exit(EXIT_FAILURE);
9     }
10
11     for (v = 0; v < V; v++) G->adj[v] = NULL;
12
13     return G;
14 }
```


Inserção de Aresta

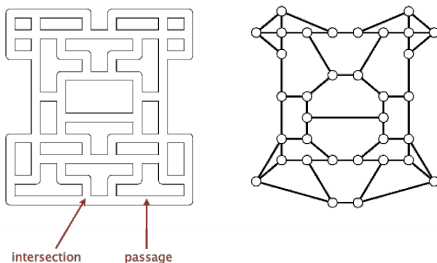
```
1 void GRAPHinsertE(Graph G, Edge e) {
2     int v = e.v;
3     int w = e.w;
4     G->adj[v] = NEW(w, G->adj[v]);
5     G->adj[w] = NEW(v, G->adj[w]);
6     G->E++;
7 }
```

Busca em Profundidade

Exploração em Labirintos

Grafo de um Labirinto:

- Vértice = interseção;
- Aresta = passagem.



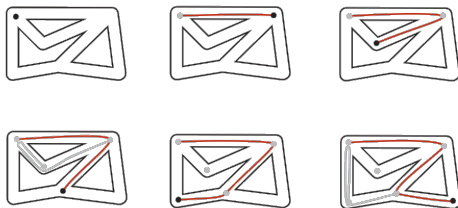
Fonte: (SEDGEWICK; WAYNE, 2014)

Objetivo Explorar cada interseção (vértice) do labirinto.

Exploração em Labirintos

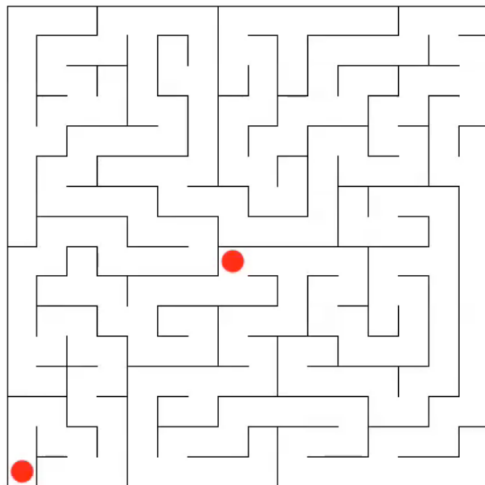
Algoritmo possível

- Desenrolar um novelo de lã atrás de você;
- Assim, marcamos cada interseção visitada e cada passagem visitada;
- Voltar pelo fio de lã quando ficar sem saída para última interseção com passagens não visitadas.



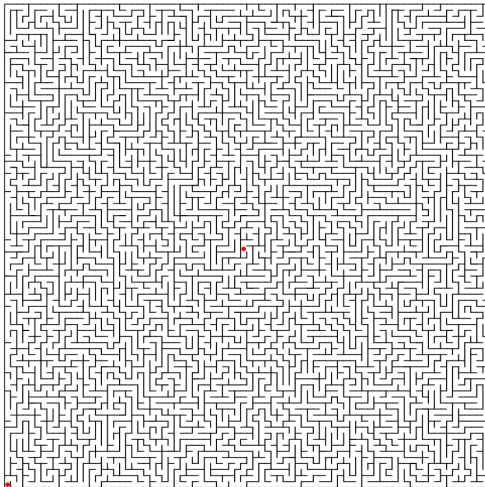
Fonte: (SEdgeWICK; WAYNE, 2014)

Labirinto Fácil



Fonte: (SEDGEWICK; WAYNE, 2014)

Labirinto Difícil



Fonte: (SEdgeWICK; WAYNE, 2014)

Busca em Profundidade

Objetivo Percorrer o grafo sistematicamente.

Ideia Imitar a exploração de labirintos (pilha de funções como novelo de lã).

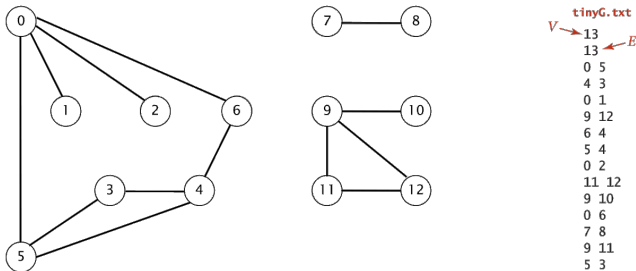
Algorithm 1: Depth-first search (Busca em Profundidade)

- 1 **def** *DFS(visitar um vertice v):*
 - 2 | Marcar v como visitado
 - 3 | Recursivamente visitar todos os vizinhos não marcados de v
-

Aplicações Típicas:

- Encontrar todos os vértices conectados a um dado vértice;
- Encontrar um caminho entre dois vértices.

Exemplo Busca em Profundidade

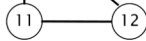
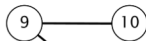
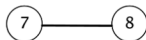
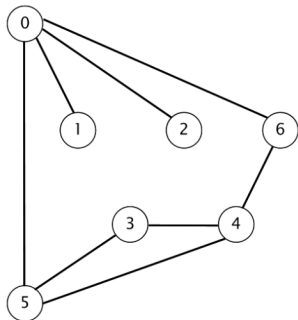


Fonte: (SEdgeWICK; WAYNE, 2014)

Estruturas auxiliares

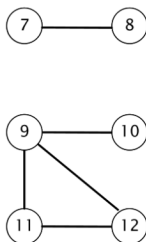
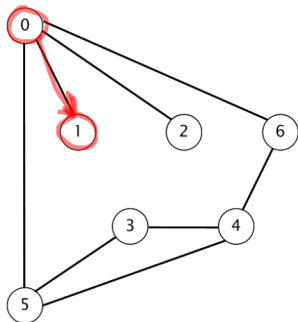
- Array `marked[]` para marcar vértices visitados;
- Array `edgeTo[]` para salvar caminhos:
 - $(\text{edgeTo}[w] == v)$ significa que a aresta $v - w$ foi utilizada para visitar w pela primeira vez.

Exemplo Busca em Profundidade



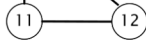
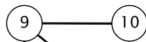
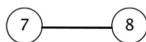
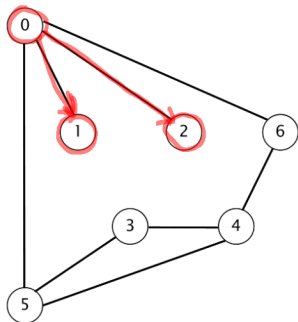
V	marked	edgeTo
0		
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		

Exemplo Busca em Profundidade



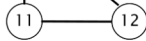
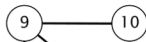
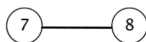
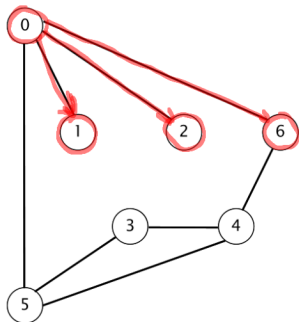
V	marked	edgeTo
0	T	
1	T	0
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		

Exemplo Busca em Profundidade



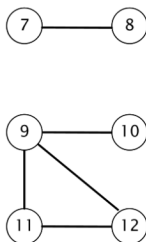
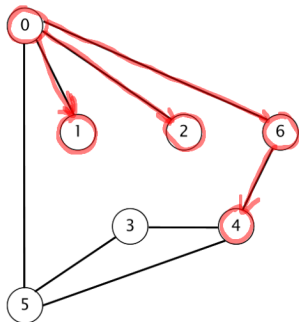
V	marked	edgeTo
0	T	
1	T	0
2	T	0
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		

Exemplo Busca em Profundidade



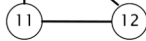
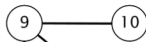
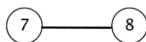
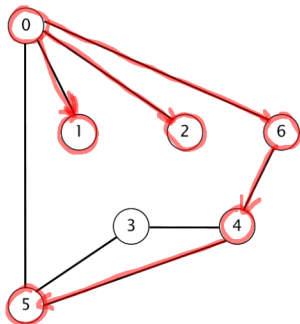
V	marked	edgeTo
0	T	
1	T	0
2	T	0
3		
4		
5		
6	T	0
7		
8		
9		
10		
11		
12		

Exemplo Busca em Profundidade



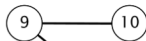
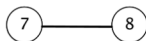
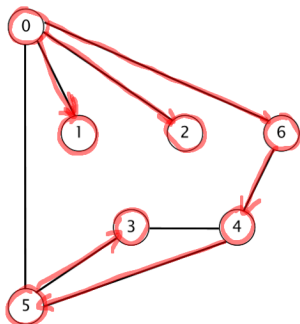
V	marked	edgeTo
0	T	
1	T	0
2	T	0
3		
4	T	6
5		
6	T	0
7		
8		
9		
10		
11		
12		

Exemplo Busca em Profundidade



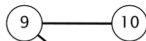
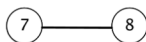
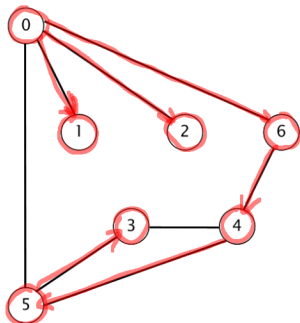
V	marked	edgeTo
0	T	
1	T	0
2	T	0
3		
4	T	6
5	T	4
6	T	0
7		
8		
9		
10		
11		
12		

Exemplo Busca em Profundidade



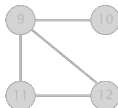
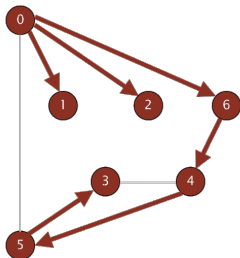
V	marked	edgeTo
0	T	
1	T	0
2	T	0
3	T	5
4	T	6
5	T	4
6	T	0
7		
8		
9		
10		
11		
12		

Exemplo Busca em Profundidade



V	marked	edgeTo
0	T	
1	T	0
2	T	0
3	T	5
4	T	6
5	T	4
6	T	0
7	F	-
8	F	-
9	F	-
10	F	-
11	F	-
12	F	-

Exemplo Busca em Profundidade



v	marked[]	edgeTo[]
0	T	-
1	T	0
2	T	0
3	T	5
4	T	6
5	T	4
6	T	0
7	F	-
8	F	-
9	F	-
10	F	-
11	F	-
12	F	-

vertices reachable from 0

Fonte: (SEdgeWICK; WAYNE, 2014)

Depois da DFS é possível

- Verificar se o vértice v é conectado a outro vertice s em tempo constante;
- Encontrar caminho o caminho $v - s$ em tempo proporcional ao comprimento do caminho.

Busca em Largura

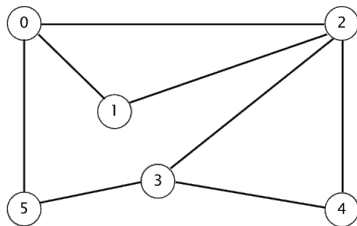
Ideia

Percorrer o grafo seguindo uma **fila** e não uma **pilha** como na busca em profundidade.

Algorithm 2: Breadth-first search (Busca em Largura)

```
1 def BFS(visitar um vertice  $v$ ):  
2   Inserir  $v$  na fila  
3   while Fila não vazia do  
4     Remover  $v$  da fila  
5     Inserir na fila todos os vizinhos não marcados de  $v$  e  
       marcá-los
```

Exemplo Busca em Largura



tinyCG.txt

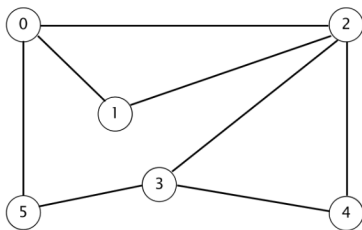
$V \rightarrow$ 6
 8 $\leftarrow E$
 0 5
 2 4
 2 3
 1 2
 0 1
 3 4
 3 5
 0 2

Fonte: (SEdgeWICK; WAYNE, 2014)

Estruturas auxiliares

- Array `distTo[]` para distâncias a partir de v ;
- Array `edgeTo[]` para salvar caminhos:
 - $(\text{edgeTo}[w] == v)$ significa que a aresta $v - w$ foi utilizada para visitar w pela primeira vez.

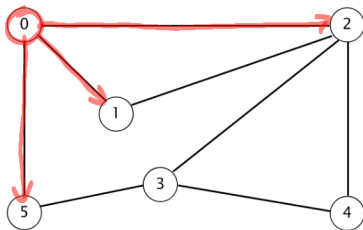
Exemplo Busca em Largura



V	edgeTo	distTo
0		
1		
2		
3		
4		
5		

fila: 0

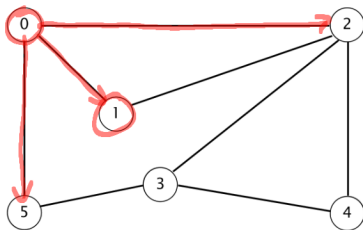
Exemplo Busca em Largura



V	edgeTo	distTo
0	-	0
1	0	1
2	0	1
3		
4		
5	0	1

fila: ~~0~~ 1 2 5

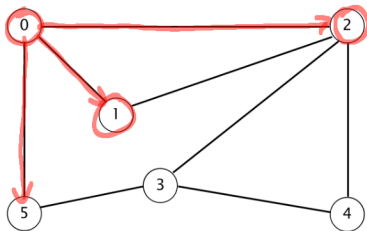
Exemplo Busca em Largura



V	edgeTo	distTo
0	-	0
1	0	1
2	0	1
3		
4		
5	0	1

fila: ~~0~~ ~~1~~ 2 5

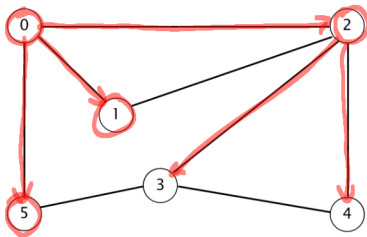
Exemplo Busca em Largura



V	edgeTo	distTo
0	-	0
1	0	1
2	0	1
3	2	2
4	2	2
5	0	1

fila: ~~0~~ ~~1~~ ~~2~~ 5 3 4

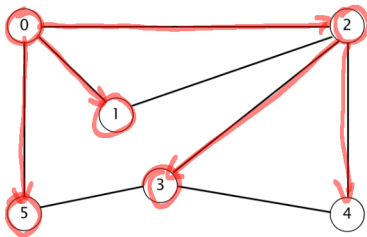
Exemplo Busca em Largura



V	edgeTo	distTo
0	-	0
1	0	1
2	0	1
3	2	2
4	2	2
5	0	1

fila: ~~0~~ ~~1~~ ~~2~~ ~~5~~ 3 4

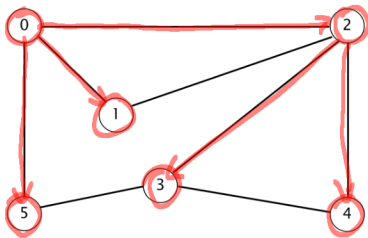
Exemplo Busca em Largura



V	edgeTo	distTo
0	-	0
1	0	1
2	0	1
3	2	2
4	2	2
5	0	1

fila: ~~0~~ ~~1~~ ~~2~~ ~~5~~ ~~3~~ 4

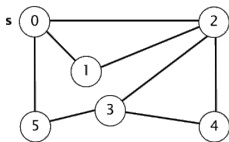
Exemplo Busca em Largura



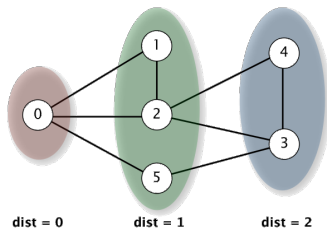
V	edgeTo	distTo
0	-	0
1	0	1
2	0	1
3	2	2
4	2	2
5	0	1

fila: ~~0~~ ~~1~~ ~~2~~ ~~5~~ ~~3~~ ~~4~~

Cálculo de Menor Caminho por Busca em Largura



graph G



Fonte: (SEdgeWICK; WAYNE, 2014)

Componentes Conexas

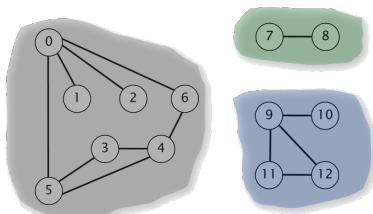
Definição

A relação “é conectado à” é uma relação de equivalência:

- Reflexiva: v é conectado à v ;
- Simétrica: se v é conectado à w , então w é conectado à v ;
- Transitiva: se v é conectado à w e w é conectado à x , então v é conectado à x ;

Definição Uma **componente conexa** é um conjunto máximo de vértices (não é possível adicionar mais nenhum ao conjunto).

Exemplo

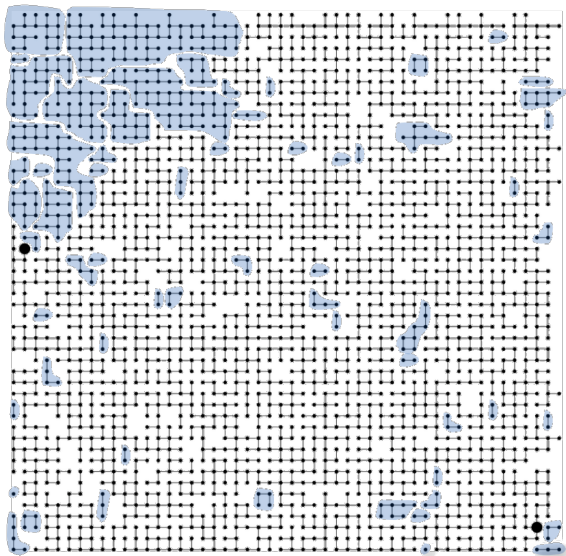


3 connected components

v	id[]
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	1
8	1
9	2
10	2
11	2
12	2

Fonte: (SEdgeWICK; WAYNE, 2014)

Exemplo



63 connected components

Conclusão

Problemas de Grafos e suas Buscas

problem	BFS	DFS	time
path between s and t	✓	✓	$E + V$
shortest path between s and t	✓		$E + V$
connected components	✓	✓	$E + V$
biconnected components		✓	$E + V$
cycle	✓	✓	$E + V$
Euler cycle		✓	$E + V$
Hamilton cycle			$2^{1.657 V}$
bipartiteness	✓	✓	$E + V$
planarity		✓	$E + V$
graph isomorphism			$2^{c\sqrt{V \log V}}$

Fonte: (SEDGEWICK; WAYNE, 2014)

Referências utilizadas na apresentação



SEDGEWICK, R.; WAYNE, K. *Algorithms: Part I*. [S.l.]: Addison-Wesley Professional, 2014.